

AE 8883: Spacecraft Engineering I

Software Development Tools
October 5, 2022



Ebenezer Arunkumar and Mason Starr

What is Embedded Software?

"**Application-specific software** designed to perform **specific tasks** repeatedly **without user intervention**. With the exception of the simple reset process, once the system is loaded, embedded software is expected to **run by itself** for a very long time **without any damage to the software**. Software developers must guarantee the **reliability, safety and correctness** of the embedded software."

- Kai Qian – Embedded Software Development with C

Key Features:

- Real-Time
- Reliability
- Usability
- Efficiency
- Adaptability
- Transferability

Embedded Software Development

Common Embedded Software Languages: Embedded Software Environments:

- C++
- C
- Assembly

- Linux
- Bare-metal(No OS)

Less common embedded software:

- Python
- C#



Case Study – Software Development for VISORS

Payload Avionics Interface Board

- Microcontroller: Onion Omega 2S+
 - Runs a version of Linux OS
- Chosen software architecture:
 - F Prime – written in C++
- Build Tools:
 - C Compiler: `mipsel-openwrt-linux-gcc`
 - C++ Compiler: `mipsel-openwrt-linux-g++`

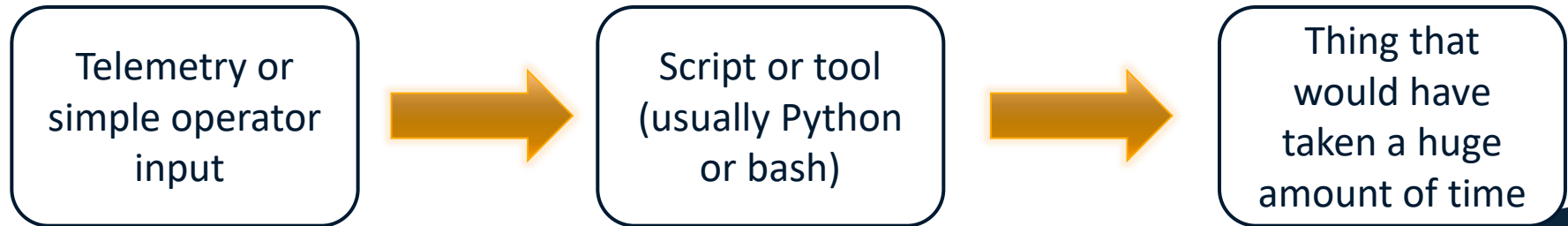


What do we need to develop software?

- Linux VM
 - To compile software for the Onion
- Python Virtual Environment
 - To safely install tools needed for F Prime
- Integrated Development Environment(IDE)
 - To see and edit our files
- Git
 - To version control our files
- Terminal
 - To interface with the VM and compile software

Case Study – Software Development for Lunar Flashlight

- Lunar Flashlight flight software already written – no access
- Operations tools delivered by JPL are not sufficient to perform operations – had to develop our own!
- We need scripts and code for:
 - Product and file generation
 - Networking: port forwarding, file transfer, data streams, etc.
 - Automatic interfaces with Box and cloud storage sites
 - Scripts that bypass commanding software to directly operate the spacecraft
 - Generation and checking of Lunar Flashlight command sequences
 - Ways to save time!



Linux Virtual Machine

- Often needed to compile software for embedded systems
 - Due to the toolchains only working on Linux
- Allows for standardized software development environment
 - Great for small and large teams!
 - Helps diagnose issues
- Great for running open source software
- Large set of free software development tools
- Tracking down environment issues is easier on Linux

Common Linux VM's

- Virtualbox
- Windows Subsystem for Linux(WSL)



Python Virtual Environment

- Many projects require python tools
- Allows users to customize their environment for the project they are working on
 - Different projects might require different versions of python libraries-> use multiple python virtual environments
- Prevents users from accidentally tampering with the system level python that come pre-installed with your operating system. This is very easy to do!
- Errors will be easier to reproduce
 - Can only be caused by the tools installed in your virtual environment, NOT system level tools
- Easy to set up
 - Creation: `python3 -m venv <venv-name-here>`
 - Activation: `<venv-name-here>/bin/activate`
 - Deactivation: `deactivate`

Choosing an IDE

- It turns out, you can write code outside of Matlab!
- IDE – Integrated Development Environment
- Lab favorite is Visual Studio Code
 - Free, open source
 - Community plugins
 - Highly customizable
 - Supports all types of languages and filetypes
 - Integrated terminal
 - Integrated git support



Example VSCode Workspace

dn2eu.csv - Untitled (Workspace) - Visual Studio Code

File Edit Selection View Go Run Terminal Help

EXPLORER

UNTITLED (WORKSPACE)

src

lfops

__pycache__

assets

__pycache__

ampcs

gcm

dictionary.properties

GdsUserConfig.xml

mission.properties

ProductWatcherConfig.xml

vm-setup.sh

vm-source.txt

mpseditor

schema

sequenceComponents

slincii

__init__.py

lfps-gui_CentOS7

sclksctet.875

models

scripts

dict

AlarmCatalog.csv

cmd_dict_tmpl.html

dictionary2html.py

dn2eu.csv

editFswDicts.ipynb

tlm_dict_tmpl.html

xml_check.py

gds

nav

sclksctet

seqgen

components

from_met.py

lfps_command_definitions.py

seqGen.ipynb

strategic

OUTLINE

TIMELINE

seqGen.ipynb

src > lfops > scripts > seqgen > seqGen.ipynb > M*Generate Lunar Flashlight Sequences > # take in contact times and type of contact, spit out sequence

Code Markdown Run All Clear Outputs of All Cells Outline Python 3.8.10 64-bit

Generate Lunar Flashlight Sequences

```
# take in contact times and type of contact, spit out sequence
import sys
from lxml import etree
import re
import importlib
import numpy as np
from datetime import datetime, timedelta

abstimeformat = 'A%-Y-%jT%H:%M:%S'
reltimeformat = 'R%H:%M:%S'

import lfops
import lfops.models.commandClasses as cc

# Make components
class Component:
    def __init__(self, template):
        self.template = template
        self.files = []

    # get the required arguments, denoted in template by <arg>
    # regex <.*?> explanation: brackets around any characters, non-greedy
    self.required_args = [x.strip('<>') for x in re.findall('<.*?>', template)]

    def aras2txt(self, **kwargs):
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL JUPYTER

mstarr32@ae-glr-407062:/telemetry/python-lfops/src\$

src > lfops > scripts > dict > dn2eu.csv

```
1 This spreadsheet contains the DN->EU conversions that get at
2 "There are two types of conversions: polynomials and enum t
3 Polynomial conversion is of the form: c0 + c1*x + c2*x^2
4 Table conversions are given in space delimited pairs: 1 YES
5
6 Notes,Conversion Type,Channel,Args,
7 XACT conversions,poly,XACT-0000,0,0.5,0,12.6g,km,
8 ,poly,XACT-0001,0,0.5,0,12.6g,km,
9 ,poly,XACT-0002,0,0.5,0,12.6g,km,
10 ,poly,XACT-0003,0,0.000005,0,12.6g,km/s,
11 ,poly,XACT-0004,0,0.000005,0,12.6g,km/s,
12 ,poly,XACT-0005,0,0.000005,0,12.6g,km/s,
13 ,poly,XACT-0006,0,0.5,0,12.6g,km,
14 ,poly,XACT-0007,0,0.5,0,12.6g,km,
15 ,poly,XACT-0008,0,0.5,0,12.6g,km,
16 ,poly,XACT-0009,0,0.5,0,12.6g,sec,
17 ,poly,XACT-0010,0,0.5,0,12.6g,sec,
18 ,poly,XACT-0035,0,5E-10,0,1.6g,N/A,
19 ,poly,XACT-0036,0,5E-10,0,1.6g,N/A,
20 ,poly,XACT-0037,0,5E-10,0,1.6g,N/A,
21 ,poly,XACT-0038,0,5E-10,0,1.6g,N/A,
22 ,poly,XACT-0039,0,5E-10,0,12.6g,rad,
23 ,poly,XACT-0040,0,5E-10,0,12.6g,rad,
24 ,poly,XACT-0041,0,5E-10,0,12.6g,rad,
25 ,poly,XACT-0042,0,0.00000005,0,12.6g,rad/s,
26 ,poly,XACT-0043,0,0.00000005,0,12.6g,rad/s,
27 ,poly,XACT-0044,0,0.00000005,0,12.6g,rad/s,
28 ,poly,XACT-0045,0,0.00000005,0,12.6g,rad/s,
29 ,poly,XACT-0046,0,0.00000005,0,12.6g,rad/s,
30 ,poly,XACT-0047,0,0.00000005,0,12.6g,rad/s,
31 ,poly,XACT-0053,0,5E-10,0,12.6g,N/A,
32 ,poly,XACT-0054,0,5E-10,0,12.6g,N/A,
33 ,poly,XACT-0055,0,5E-10,0,12.6g,N/A,
34 ,poly,XACT-0056,0,5E-10,0,12.6g,N/A,
35 ,poly,XACT-0057,0,0.00004,0,12.6g,rad/s,
36 ,poly,XACT-0059,0,0.001,0,12.6g,rad,
```

bash - src

Col 3: Ln 31, Col 7 Spaces: 4 UTF-8 with BOM LF CSV

Version Control with Git

Git: Do I NEED to use it?

What is Git?

- **Version control** software designed to keep track of source code

What is version control?

- Keeping track of **not just the current state** of the files, but also **every previous version**

Why do I need it?

- If you mess up and break something, you can go back to the last working version and see what you changed

Git is designed to be used by dozens, hundreds, or even thousands of people working **simultaneously** on the same files **without horrible conflicts and collisions**.

How does this work? We'll get to it later.



Using Github

Github Locations

- GT Research Github: <https://github-research.gatech.edu>
 - Proprietary GT ITAR code
- GT Regular Github: <https://github.gatech.edu>
 - Proprietary GT code
- Github proper: <https://github.com>
 - Personal code
 - Open source projects

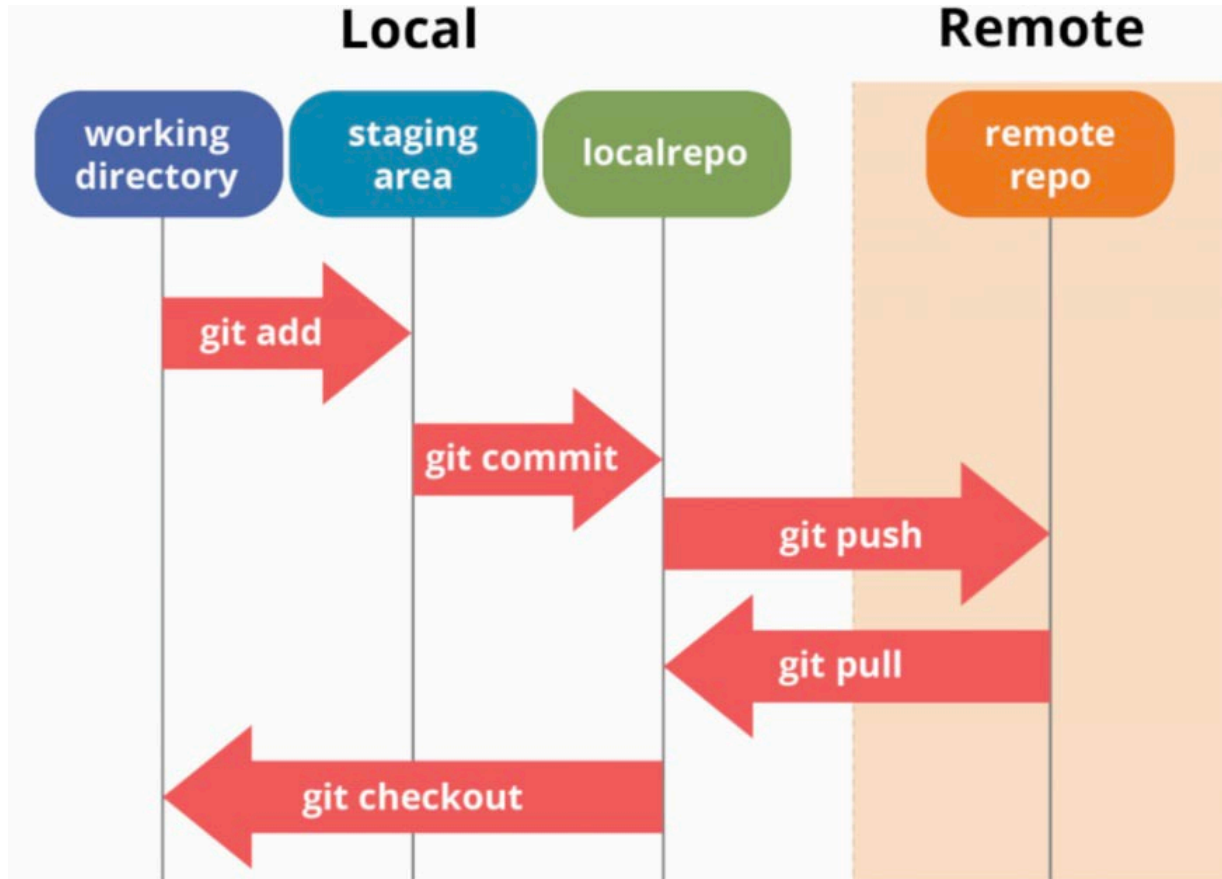
Github Repository(repo): A collection of source code with a common theme

When you're performing an action with git, you act on the entire **repo**:

- if you edit one file and commit it, you commit the entire **repo**.
- If you want to download some files, you download the entire **repo**.
- Etc.



BASIC Git Workflow



Git & Commits

Commit: The basic unit of code change on a git repo

- Generates a snapshot of the code as it exists in that moment, preserved for all time
- Each commit has an identifying hash and message to understand the included changes
 - **Bad:** "Fixed some things"
 - Good: "Changed sliding mode controller to use std vector"
 - **Bad:** "Bug fixes"
 - Good: "Fixed UART segfaulting when given a bad port"
 - **Bad:** "Not sure what this is, committing it to switch branches"

When to commit?

- A commit should be the minimum amount of change that is useful and complete
- If your commit messages are super long -> commits are too large
- If you're committing things that you haven't tested -> committing too often

Git & Branches

Branch: A unique set of code changes on the git repo with a unique name

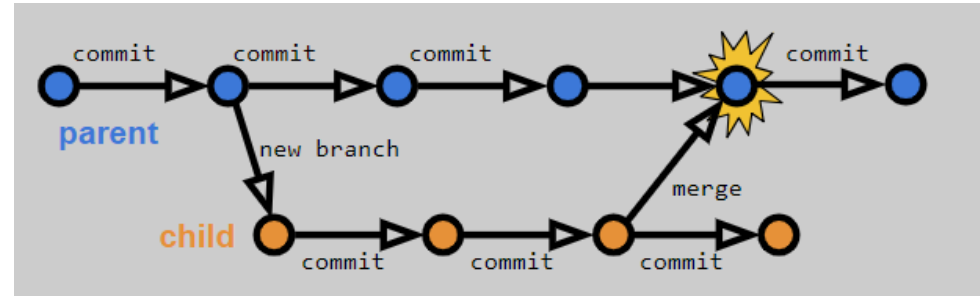
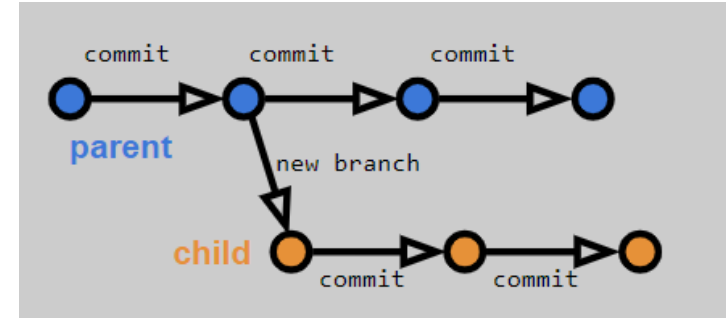
- Default Branch: main/master
 - Cardinal Rule: Do not commit directly to master – ALWAYS use branches

Making commits to a branch **does not** affect any other branches

- Allows for concurrent development by multiple members

Once your code is tested, you can *merge* with the main/master branch

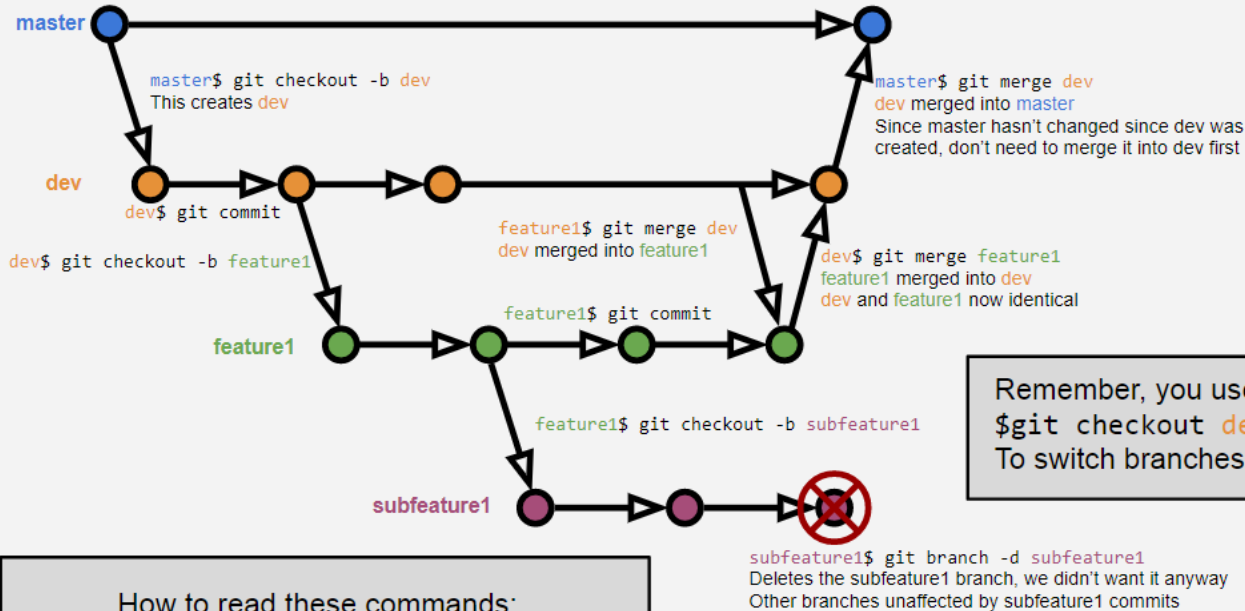
- Multiple people can merge into master at the same time
- git is pretty good with resolving merge conflicts



Git & Branches

Terry's Guide to Git Branches

Start with `master` branch, you get this when you create the repo



How to read these commands:

`dev$ git checkout -b feature1`

↑
Current branch

← This is what you type →

This is a commit, they're created either by committing changes to a branch or by merging two branches

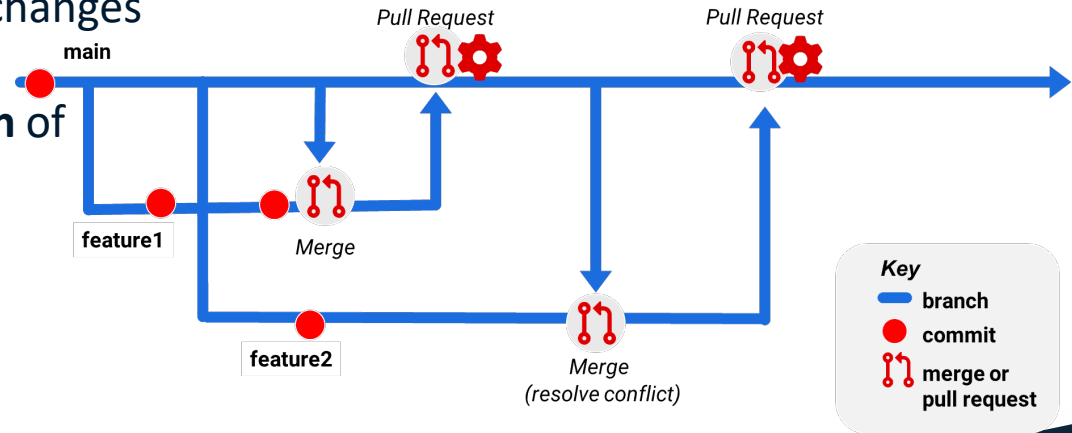
Software Review

In order to ensure the robustness and safety of the git repo, all code merging into devel or main/master **must** be reviewed.

- Helps ensure that all code in devel and main/master are tested and production ready

Pull Requests

- Allows you to tell others about changes you've pushed to a branch
- Allows for **review and discussion** of changes with collaborators



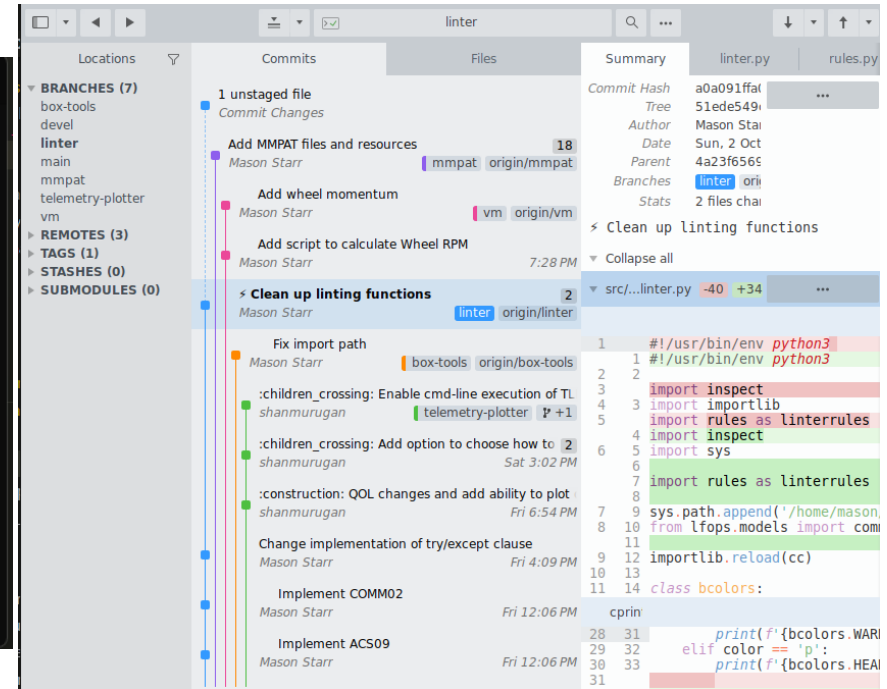
Using Git

Command line interface

```
mason@ASUSPrimeZ390-P:~/python-lfops$ git status
On branch devel
Your branch is up to date with 'origin/devel'.

nothing to commit, working tree clean
mason@ASUSPrimeZ390-P:~/python-lfops$ git branch
* devel
  linter
  main
  mmpat
  telemetry-plotter
  vm
mason@ASUSPrimeZ390-P:~/python-lfops$ git checkout linter
Switched to branch 'linter'
Your branch is up to date with 'origin/linter'.
mason@ASUSPrimeZ390-P:~/python-lfops$
```

GUI – Sublime Merge



Git General Advice

1. When you first sit down to start working, **pull** the remote repo
2. New features = new branches
3. Don't mess with other people's branches
4. You can undo *most* git mistakes, but always ask for help!
5. If you get a lot of git errors, don't try things randomly -> ask for help!
6. If you want to try something out, make a **new** branch and go crazy

Resources:

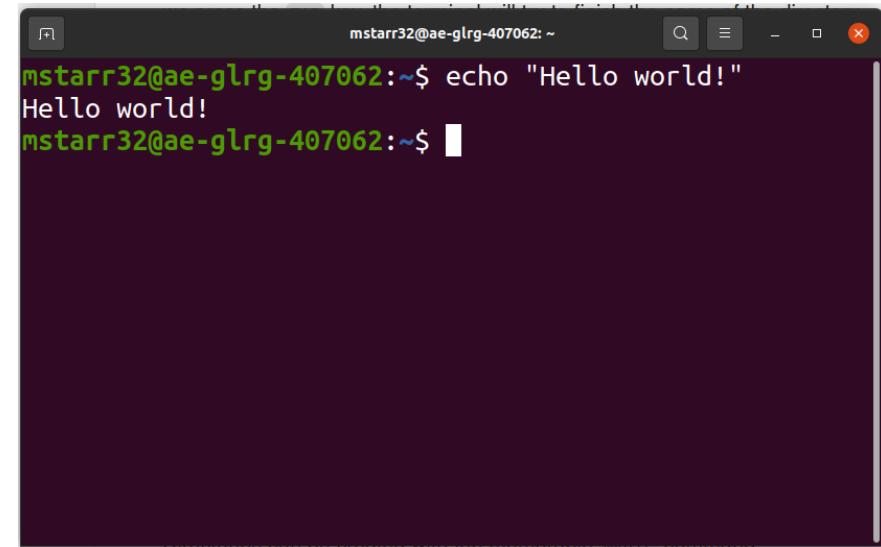
Git Cheat sheet: <https://education.github.com/git-cheat-sheet-education.pdf>

Git **Simple** Guide: <https://rogerdudler.github.io/git-guide/>

Intro to Linux and the Command Line

What is the Terminal?

- Text based interface to your computer – computers did not always use a mouse!
- Comes stock on Linux and MacOS, Windows can use WSL.
- Lots of shells to choose from – bash is the most common.

A screenshot of a terminal window with a dark purple background. The window title bar shows 'mstarr32@ae-glr-407062: ~'. The terminal displays a green prompt 'mstarr32@ae-glr-407062:~\$' followed by the command 'echo "Hello world!"'. The output 'Hello world!' is shown on the next line. A second green prompt 'mstarr32@ae-glr-407062:~\$' is followed by a white cursor. At the bottom of the terminal, there is a small line of text: 'Directories can be created with the mkdir command.'

Why do I care?

- High degree of control
 - Script testing
 - File management
 - Great for troubleshooting
 - Interfacing with legacy software
 - Networking
- Able to *ssh* into other computers
- Super fast, once you get used to it
- Very useful skill to have



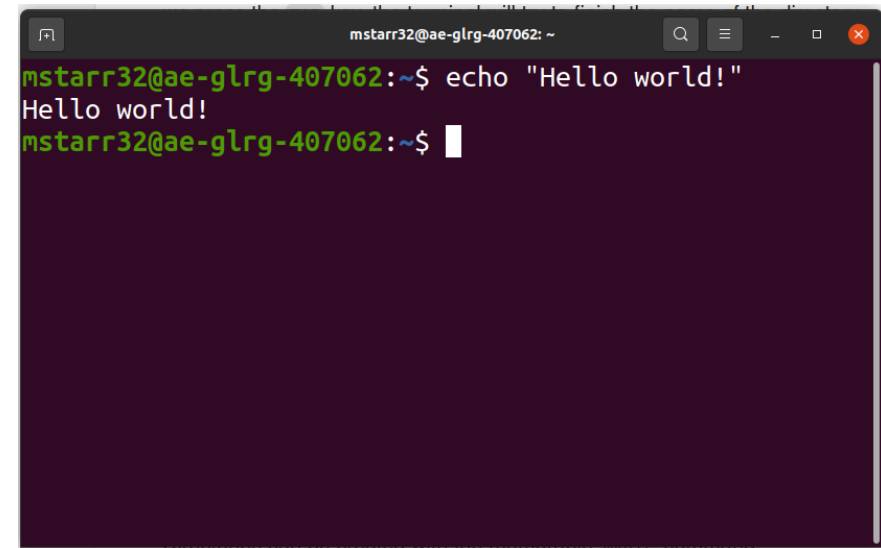
How to set up?

- If you have a Linux or Mac, congrats, you are ready to go.
- If you have a Windows machine:
 - Powershell is the default shell. Very different than Linux shells.
 - Install Windows Subshell for Linux (WSL)
 - Install Ubuntu Linux from the app store
 - Install gWSL from the app store
 - Install Microsoft Terminal from the app store
 - Follow some online guides if you get stuck!
- Or, dual boot your computer with Linux!



Key concepts

- Directory based
 - . = current directory
 - .. = one directory up
 - ~ = home directory
- Relative paths: relative to current directory
 - ../../testing/
 - Desktop/presentation.pptx
- Absolute paths: start with a /
 - /home/mstarr/
 - /mnt/c/Users/
- Do things by running commands
 - Stock or user defined
 - Commands can take arguments

A terminal window with a dark purple background and green text. The window title is 'mstarr32@ae-glr-407062: ~'. The prompt is 'mstarr32@ae-glr-407062:~\$'. The user has entered the command 'echo "Hello world!"' and the output is 'Hello world!'. The prompt is now 'mstarr32@ae-glr-407062:~\$' with a cursor.

```
mstarr32@ae-glr-407062:~$ echo "Hello world!"
Hello world!
mstarr32@ae-glr-407062:~$
```

Basic commands

Command	Function
ls	List files in directory
cd	Change directory
pwd	Present working directory
cat	Concatenate / print
vi/nano	Edit file
mkdir	Make directory
touch	Make a new empty file
git	Interface with github
ssh	Tunnel to another computer on the network

Command	Function
cp	Copy a file
mv	Move a file
sudo	Run a command as a super user
man	Get help on a command
ip a	See all your IP settings
ps aux	List all active processes

Whatever else you want!

Examples

```
mstarr32@ae-glr-407062: ~  
mstarr32@ae-glr-407062:~$ pwd  
/home/mstarr32  
mstarr32@ae-glr-407062:~$ ls  
bin          Pictures          Templates  
Desktop      Public           test_env  
Documents    requirements.txt  Videos  
Downloads    revised_trunc_sms.txt  
Music        snap  
mstarr32@ae-glr-407062:~$ cd Documents/  
mstarr32@ae-glr-407062:~/Documents$ ls  
mstarr32@ae-glr-407062:~/Documents$ cd ..  
mstarr32@ae-glr-407062:~$
```

Examples

```
mstarr32@ae-glr-406015: ~  
mstarr32@ae-glr-406015: ~  
mstarr32@ae-glr-407062: ~  
mstarr32@ae-glr-407062:~$ ssh mstarr32@ae-glr-406015  
#####  
# Georgia Institute of Technology - Terms of use #  
# #  
# This computer system is the property of the Georgia Institute #  
# of Technology. Any user of this system must comply with all #  
# Institute and Board of Regents policies, including the #  
# Acceptable Use Policy, Cyber Security Policy and Data Privacy #  
# Policy (http://b.gatech.edu/it-policies). Users should have #  
# no expectation of privacy, as any and all files on this #  
# system may be intercepted, monitored, recorded, copied, #  
# inspected, and/or disclosed to authorized personnel in order #  
# to meet Institute obligations. #  
# #  
# By using this system, I acknowledge and consent to these #  
# terms. #  
#####  
Password:  
Welcome to Ubuntu 20.04.4 LTS (GNU/Linux 5.15.0-46-generic x86_64)
```

Examples

```
mstarr32@ae-glg-406015: ~/testing/new-directory
mstarr32@ae-glg-406015: ~/testing/new-directory
mstarr32@ae-glg-406015:~/testing$ pwd
/home/mstarr32/testing
mstarr32@ae-glg-406015:~/testing$ ls
mstarr32@ae-glg-406015:~/testing$ mkdir new-directory
mstarr32@ae-glg-406015:~/testing$ cd new-directory/
mstarr32@ae-glg-406015:~/testing/new-directory$ ls
mstarr32@ae-glg-406015:~/testing/new-directory$ touch text
mstarr32@ae-glg-406015:~/testing/new-directory$ ls
text
mstarr32@ae-glg-406015:~/testing/new-directory$ vi text
mstarr32@ae-glg-406015:~/testing/new-directory$ cat text
I've added some text
```

Examples

```
mstarr32@ae-glg-407062: /telemetry/python-lfops/src/lfops
mstarr32@ae-glg-407062: /telemetry/python-lfops/src/lfops$ git status
On branch box-tools
Your branch is up to date with 'origin/box-tools'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  ../../pyvenv.cfg
  ../../requirements.txt
  assets/ampcs/gcm/cubesat_5.2/logs/
  assets/sequenceComponents/
  scripts/strategic/
  scripts/tactical/logs/
  scripts/tactical/old_pickles/

nothing added to commit but untracked files present (use "git add" to track)
mstarr32@ae-glg-407062: /telemetry/python-lfops/src/lfops$
```